

Creative Programming: Data Visualisation

Lecture 9

Narration

Table of contents

- Visualising Data
- More Charts and Techniques
- Infographics

Narration

Data Visualisation

Lecture 9



Narration

(Re)visiting slides?

You can jump to any slide with the menu to the (bottom) left.

Did you miss a slide or want to revisit?

Open the narration tab while studying to get an explanation of difficult slides.



Course stuff

- Exercise 4 hand-in was missing; deadline extended by 1 week
- We will be extra careful with the breaks today
- Schedule of milestone 2 will be announced soon by Thomas

Narration

Today's code

- Today we will be going over a lot more *advanced* code
 - Don't worry if you don't understand every line!
 - Focus on the overall patterns and techniques
 - Hopefully some of these examples will inspire you/help you with your project

Narration

Visualising Data

Narration

Recap: Data Sources

- Last week we learned how to load data into our sketches
 - CSV for tabular data
 - JSON for nested structures
 - APIs for live data
- This week: how to **show** that data visually

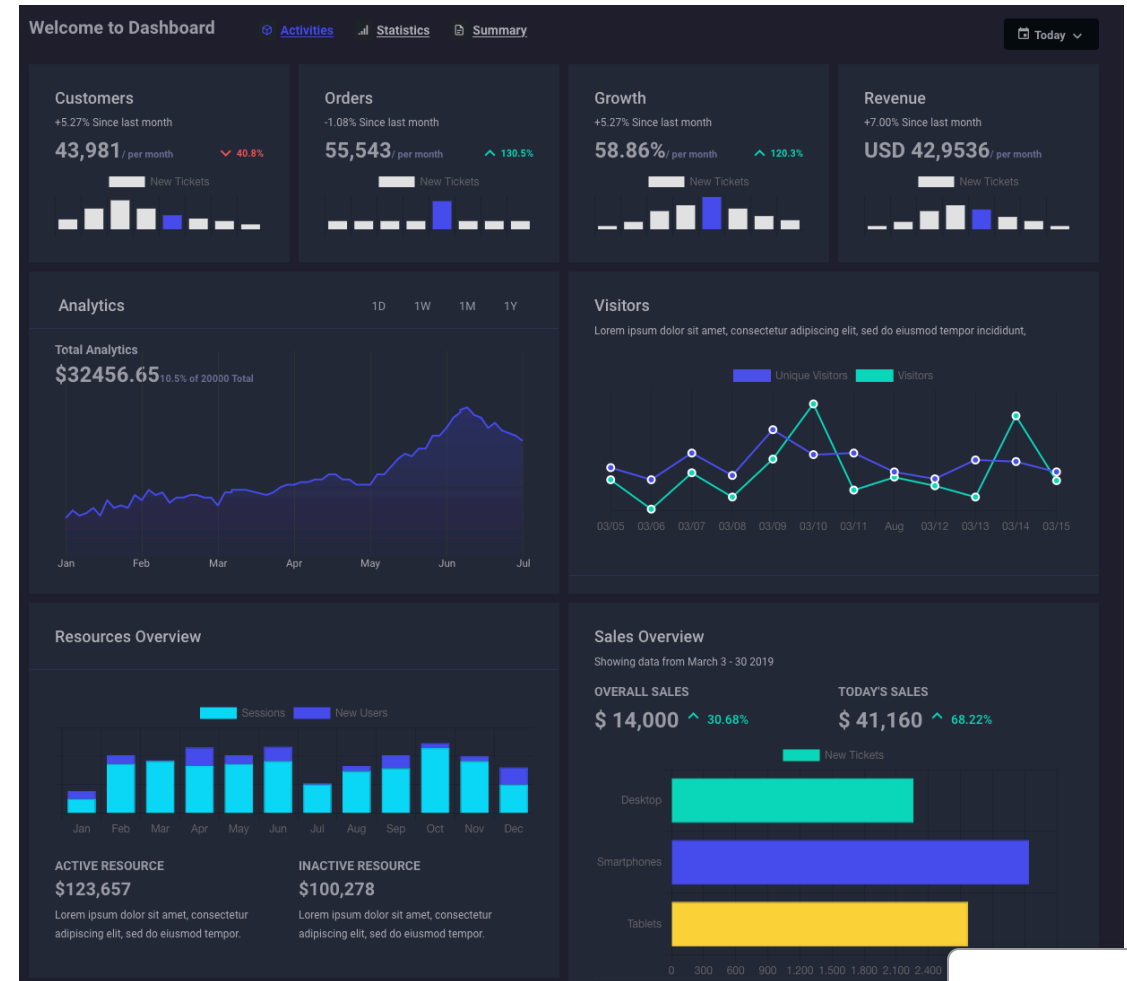
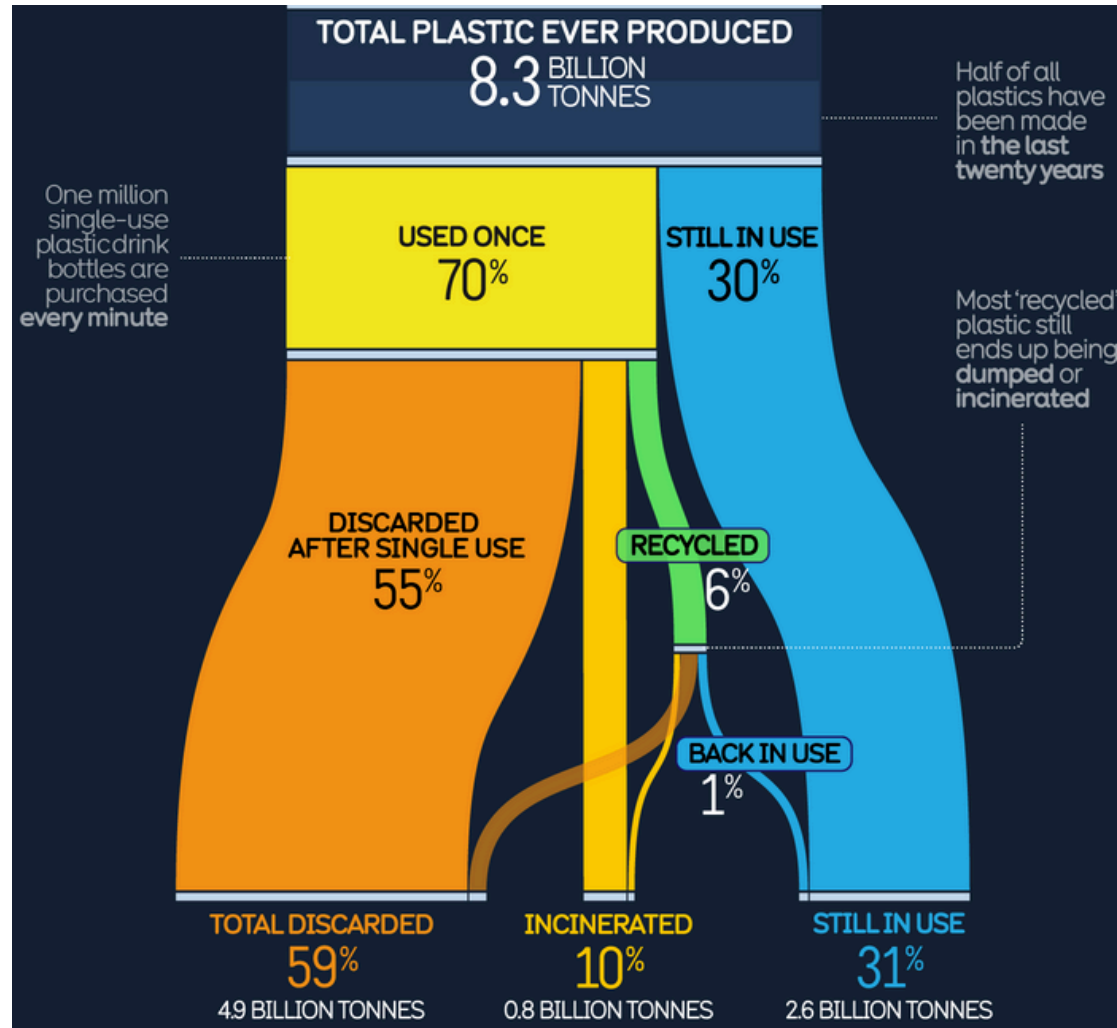
Narration

Why visualise?

- Numbers alone are hard to interpret
- Visualisation reveals patterns, trends, and outliers
- It makes data accessible and engaging
- In creative programming, the visual IS the output

Narration

Visualisation in the real world



Narration

The core idea

- Data visualisation = mapping values to visual properties
- `map()` is the key function for this
- One data value can drive one or more visual properties:
 - **Position:** x, y
 - **Size:** width, height, diameter
 - **Colour:** r, g, b, alpha
 - **Angle:** rotation, arc size

Narration

The building blocks

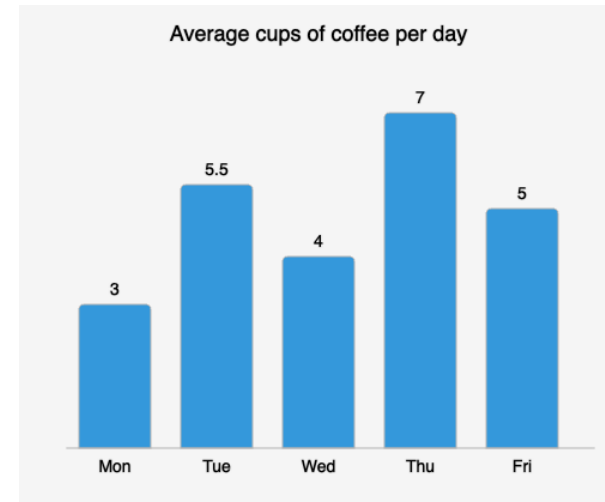
- Every chart is built from things we already know
 - Shapes: `rect()`, `circle()`, `line()`, `image()`
 - Position: x, y coordinates
 - Colour: `fill()`, `stroke()`
 - Text: `text()`, labels, titles
- The difference: these properties are **driven by data**

Narration

Bar charts

- Each bar represents a category, the height represents a value
- Loop through an array and draw a `rect()` per value
- Labels and values are drawn with `text()`

```
1 let labels = ["Mon", "Tue", "Wed", "Thu", "Fri"];
2 let values = [3.0, 5.5, 4.0, 7.0, 5.0];
3
4 function setup() {
5   createCanvas(500, 420);
6   textAlign(CENTER); // Centre text instead of left-aligning
7   noStroke();
8 }
9
10 function draw() {
11   background(245);
12
13   // Chart title
```



Narration

Key formula: positioning bars

- Calculate each bar's x position:

```
1 let x = startX + i * (barWidth + spacing);
```

- Draw from the bottom up:

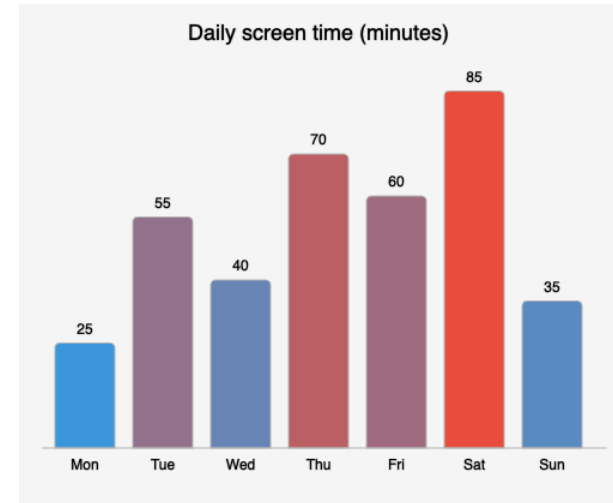
```
1 let barHeight = values[i] * scale;  
2 rect(x, bottomY - barHeight, barWidth, barHeight);
```

Narration

Adding colour as a second dimension

- Use `map()` to convert values into colours
- Higher values = warmer colours, lower values = cooler colours

```
1 let labels = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"];
2 let values = [25, 55, 40, 70, 60, 85, 35];
3
4 function setup() {
5   createCanvas(500, 420);
6   textAlign(CENTER); // Centre text instead of left-aligning
7   noStroke();
8 }
9
10 function draw() {
11   background(245);
12
13   fill("black");
14   textSize(18);
15   text("Daily screen time (minutes)", width / 2, 30);
16 }
```



Narration

Key concept: flipping y

- In p5.js, $y = 0$ is the **top** of the canvas
- In charts, higher values should be **higher** on screen
- `map()` handles this by reversing the output range:

```
1 let y = map(value, 0, maxVal, bottomY, topY);  
2 //                               high y → low y
```

Narration

Line charts

- Shows how values change over a sequence (often time)
- Draw `line()` segments between consecutive data points
- `map()` converts values to screen positions

```
1 let months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"];
2 let temps = [2, 3, 6, 10, 15, 18];
3
4 ▾ function setup() {
5   createCanvas(500, 420);
6   textAlign(CENTER);
7   noStroke();
8 }
9
10 ▾ function draw() {
11   background(245);
12
13   fill("black");
```

Narration

Chart design tips

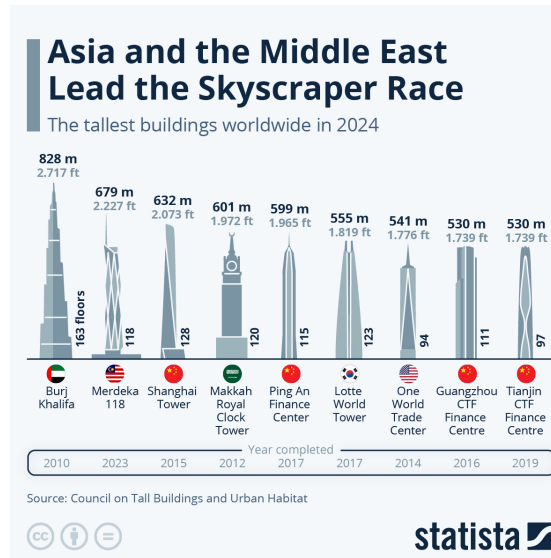
- Always label your axes and add a title
- Use colour purposefully, not decoratively
- Start bar charts at zero to avoid misleading proportions
- Keep it simple: one chart, one message
- Use `textAlign()` to position labels cleanly

Narration

Chart builders

💡 Class Assignment: Build a Chart

1. Pick either a bar chart or line chart and copy the code from the earlier slides.
2. Find or generate at least 6 data points with labels. You can make up your own data!
3. Draw the chart with proper axes, labels, and a title
4. Add colour mapping to encode an extra dimension (e.g. warm/cool colours for high/low values)



Narration

More Charts and Techniques

Narration

Hover detection for charts

- For chart interactivity, hover detection is really neat!
- It can be useful for long conditions to store them in a variable for readability

```
1 // For rectangles (bars):  
2 let isHovered = mouseX > x && mouseX < x + barWidth &&  
3     mouseY > barTop && mouseY < bottomY;  
4  
5 // For circles (scatter plots):  
6 let isHovered = dist(mouseX, mouseY, cx, cy) < radius;
```

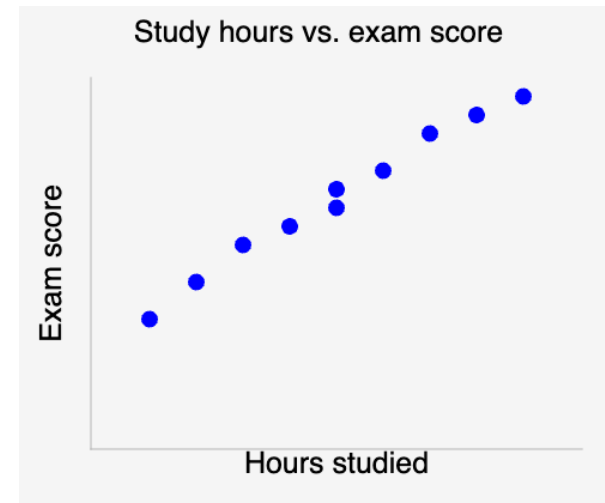
- The `dist()` function calculates the distance between two points

Narration

Scatter plots

- Shows the relationship between two variables
- Each point has an x and a y value mapped to canvas positions
- Great for seeing correlations and clusters

```
1 // Study hours vs. exam score
2 let hours = [2, 3, 5, 4, 7, 6, 8, 1, 9, 5];
3 let scores = [45, 55, 70, 60, 85, 75, 90, 35, 95, 65];
4 let names = ["A", "B", "C", "D", "E", "F", "G", "H", "I", "J"];
5
6 function setup() {
7   createCanvas(500, 420);
8   textAlign(CENTER);
9   angleMode(DEGREES);
10 }
11
12 function draw() {
13   background(245);
```

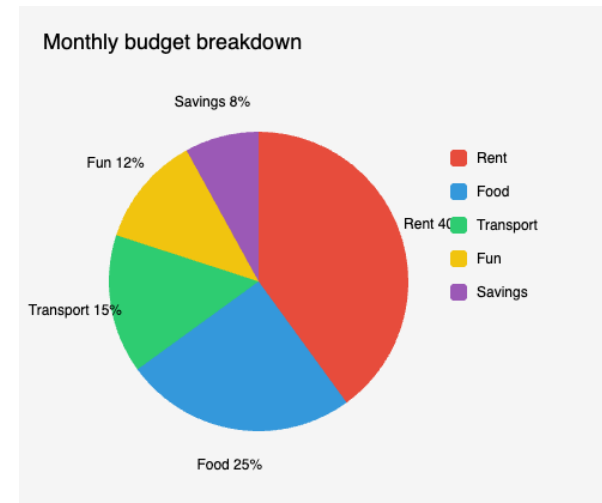


Narration

Pie charts

- Shows parts of a whole as slices of a circle
- Use `arc()` with PIE mode to draw each slice
- Angles accumulate so slices connect seamlessly

```
1 let labels = ["Rent", "Food", "Transport", "Fun", "Savings"];
2 let values = [40, 25, 15, 12, 8];
3 let colours;
4
5 ▾ function setup() {
6   createCanvas(500, 420);
7   angleMode(DEGREES);
8   ▾ colours = [
9     color(231, 76, 60),
10    color(52, 152, 219),
11    color(46, 204, 113),
12    color(241, 196, 15),
13    color(155, 89, 182)
```



Narration

Key formula: values to angles

- Remember to set `angleMode(DEGREES)` in `setup()`!

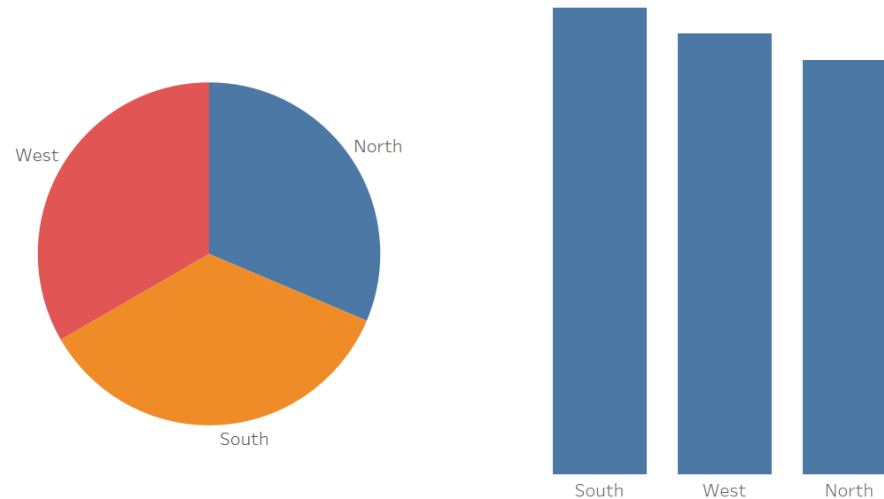
```
1 let total = 0;  
2 for (let i = 0; i < values.length; i++) {  
3   total += values[i];  
4 }  
5  
6 let sliceAngle = map(values[i], 0, total, 0, 360);
```

- The full circle is 360 degrees
- Each slice gets a fraction of the circle

Narration

A word of caution on pie charts

- Pie charts look nice, but they are **hard to read** with many slices
- Human eyes are bad at comparing angles and areas
- Works well for **two categories** (e.g. 60% vs. 40%)
- For 3+ categories, a **bar chart is almost always clearer**



Narration

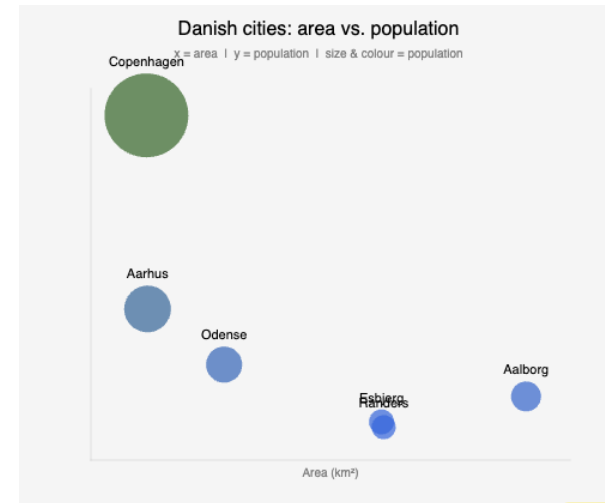
Multiple mappings

- One data point can drive multiple visual channels
- Example: position, size, AND colour from the same dataset
- Be careful not to overdo it

```

1 // Danish cities: name, area (km²), population (thousands)
2 ▾ let cities = [
3   { name: "Copenhagen", area: 88, pop: 650 },
4   { name: "Aarhus", area: 91, pop: 285 },
5   { name: "Odense", area: 304, pop: 180 },
6   { name: "Aalborg", area: 1144, pop: 120 },
7   { name: "Esbjerg", area: 742, pop: 72 },
8   { name: "Randers", area: 748, pop: 62 }
9 ];
10
11 ▾ function setup() {
12   createCanvas(500, 420);
13   textAlign(CENTER);

```

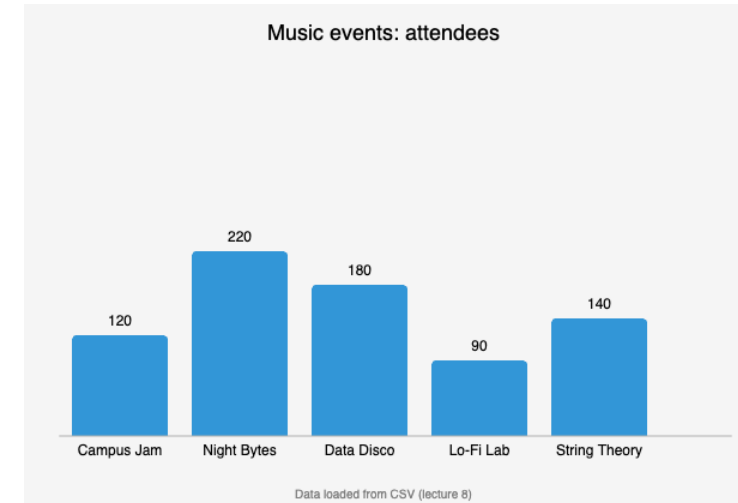


Narration

Using CSV data from last week

- Combine `loadTable()` from last week with bar chart drawing
- Loop through rows and map column values to bars

```
1 let CSV_URL = "https://raw.githubusercontent.com/Sipondo/creative-programming-2026/refs/heads/main/lecture_8/data/music_events.csv";
2 let table;
3
4 function preload() {
5   table = loadTable(CSV_URL, "csv", "header");
6 }
7
8 function setup() {
9   createCanvas(600, 420);
10  textAlign(CENTER);
11  noStroke();
12 }
13
14 function draw() {
15   background(245);
```



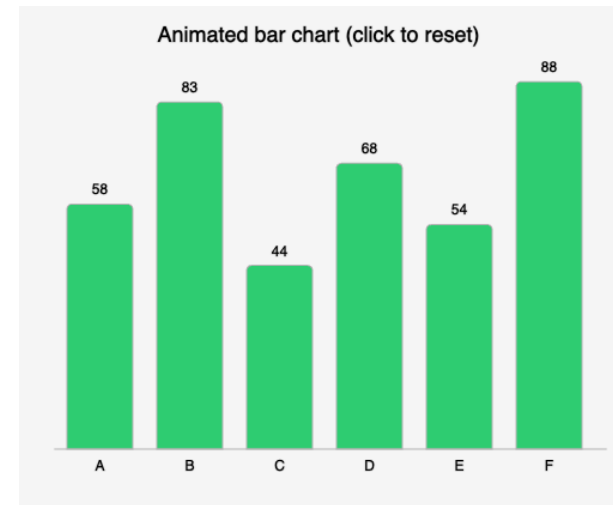
Narration

Animating charts with lerp

- We have used an easing formula before, but you can also use `lerp()`
- Use either to smoothly transition bar heights from zero

```
1 currentHeight = lerp(currentHeight, targetHeight, 0.03);
```

```
1 let labels = ["A", "B", "C", "D", "E", "F"];
2 let targetValues = [60, 85, 45, 70, 55, 90];
3 let currentValues = [0, 0, 0, 0, 0, 0];
4
5 function setup() {
6   createCanvas(500, 420);
7   textAlign(CENTER);
8   noStroke();
9 }
10
11 function draw() {
12   background(245);
13
14   fill("black");
```



Narration

Adding data on the fly

- We can add to our data arrays during runtime
- Click to add random data, press 'r' to reset

```
1 let labels = [];  
2 let values = [];  
3 let counter = 0;  
4  
5 function setup() {  
6   createCanvas(500, 420);  
7   textAlign(CENTER);  
8   noStroke();  
9 }  
10  
11 function draw() {  
12   background(245);  
13  
14   fill("black");  
15   textSize(16);  
16   text("Click to add data, press 'r' to reset", width / 2, 25);
```

Click to add data, press 'r' to reset

Click anywhere to start adding bars!

Narration

Finding data

- Good visualisations start with **interesting data**
- Some great open data sources:
 - [Danmarks Statistik](#) — population, economy, education, health
 - [Open Data DK](#) — Danish municipal open data
 - [DMI Open Data](#) — Danish weather and climate
 - [SimpleMaps Denmark](#) — city coordinates and populations
 - [Our World in Data](#) — global datasets, easy CSV downloads
- You only need 5-15 data points — keep it small!

Narration

Find your data

Class Assignment: Find Your Data

1. Browse the data sources from last slide (or find your own) and pick something interesting
2. Download or copy the data — aim for 5-15 data points with at least 2 columns
3. Load the data into p5.js (array, CSV, or JSON) and create one chart that shows something interesting
4. Pick a chart type that fits: bar for comparisons, line for trends, scatter for relationships, or a pie

We will continue with this dataset in the next assignment!



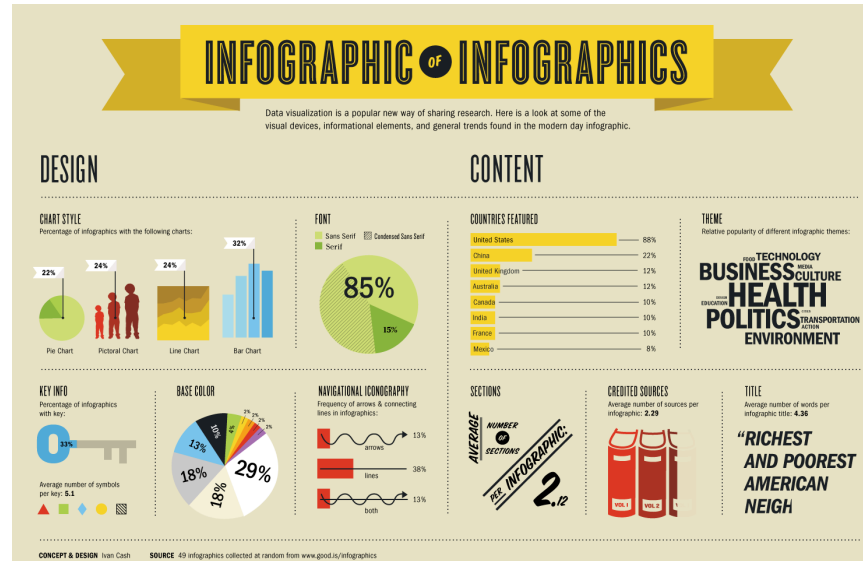
Narration

Infographics

Narration

What is an infographic?

- A visual summary that combines **multiple elements** into one layout
- Charts + icons + key numbers + text + colour
- Tells a complete **story** at a glance
- Common in journalism, reports, social media, and public health



Narration

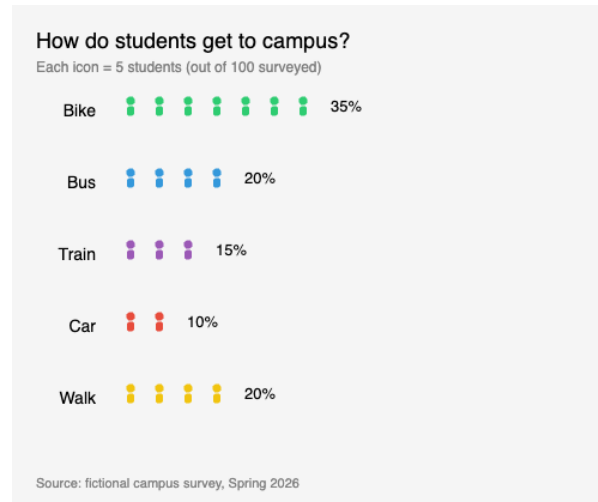
Pictogram charts

- Replace bars with **repeated icons** to represent quantities
- Each icon = a fixed number of units
- More engaging and intuitive than abstract bars

```

1  let categories = ["Bike", "Bus", "Train", "Car", "Walk"];
2  let values = [35, 20, 15, 10, 20];
3  let colours;
4
5  function setup() {
6    createCanvas(500, 420);
7    noStroke();
8    colours = [
9      color(46, 204, 113),
10     color(52, 152, 219),
11     color(155, 89, 182),
12     color(231, 76, 60),
13     color(241, 196, 15),

```



Narration

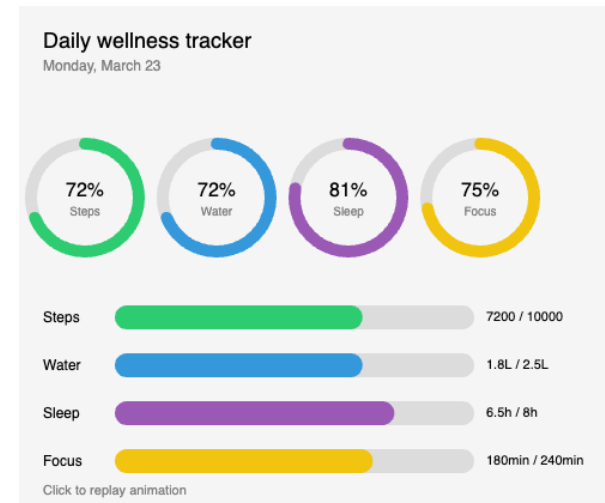
Progress and gauges

- Show progress toward a goal (current vs. target)
- Circular gauges use `arc()`, progress bars use `rect()`
- Great for dashboards and tracker-style layouts

```

1  ▾ let goals = [
2    { label: "Steps", current: 7200, target: 10000, unit: "" },
3    { label: "Water", current: 1.8, target: 2.5, unit: "L" },
4    { label: "Sleep", current: 6.5, target: 8, unit: "h" },
5    { label: "Focus", current: 180, target: 240, unit: "min" },
6  ];
7
8  let animProgress = 0;
9  let colours;
10
11 ▾ function setup() {
12   createCanvas(500, 420);
13   angleMode(DEGREES);

```



Narration

Key formula: circular gauge

- Same idea as pie charts — remember `angleMode(DEGREES)`!

```
1 let pct = current / target;           // 0.0 to 1.0
2 let endAngle = -90 + 360 * pct;        // start from top
3
4 // Background ring
5 arc(cx, cy, size, size, -90, -90 + 360);
6
7 // Filled ring
8 arc(cx, cy, size, size, -90, endAngle);
```

- `-90` starts the arc at the **top** (12 o'clock)

Narration

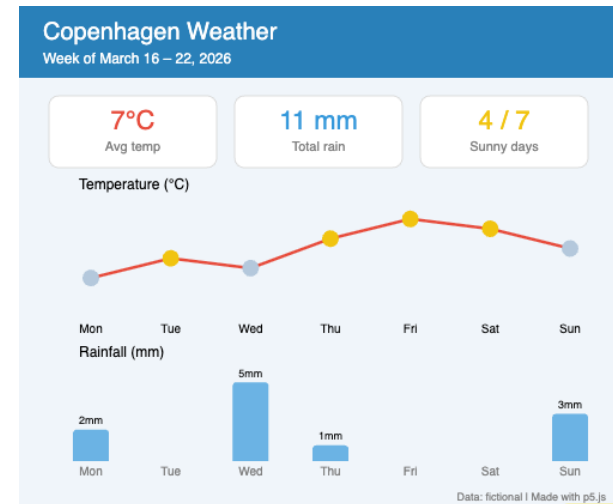
Composing an infographic

- Divide the canvas into **regions** (header, stats, charts)
- Draw each component in its own region
- Use consistent colours and typography throughout

```

1 // Data: Copenhagen weather, one week
2 let days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"];
3 let temps = [4, 6, 5, 8, 10, 9, 7];
4 let rain = [2, 0, 5, 1, 0, 0, 3]; // mm
5 let sunny = [false, true, false, true, true, true, false];
6
7 let animProgress = 0;
8
9 function setup() {
10   createCanvas(500, 420);
11   noStroke();
12 }
13

```



Narration

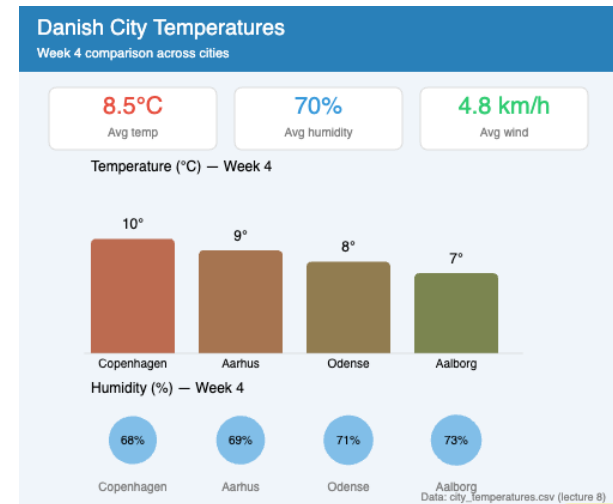
Example: Danish cities from CSV

- Uses `loadTable()` to load the same CSV from lecture 8
- Stat boxes summarise averages across cities
- Bar chart compares temperatures, dot sizes show humidity

```

1 // Danish cities infographic using CSV from lecture 8
2 let CSV_URL = "https://raw.githubusercontent.com/Sipondo/creative-programming-2026/refs/heads/main/lecture_8/data/city_temperatures.csv";
3 let table;
4 let animProgress = 0;
5
6 function preload() {
7   table = loadTable(CSV_URL, "csv", "header");
8 }
9
10 function setup() {
11   createCanvas(500, 420);
12   noStroke();

```



Narration

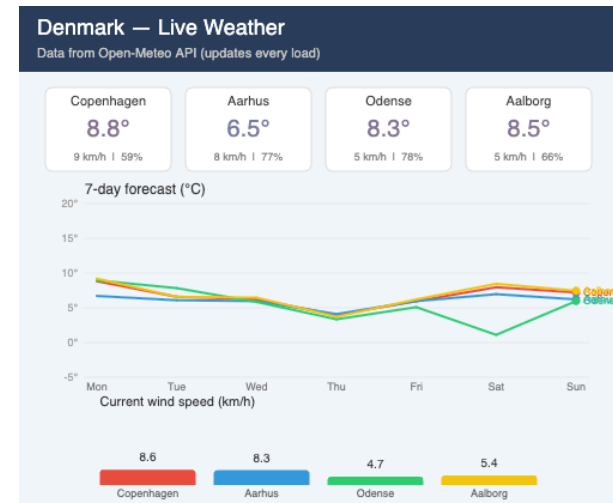
Example: live weather dashboard

- Fetches **live weather** for Copenhagen, Aarhus, Odense, and Aalborg
- Uses the Open-Meteo API from last lecture
- Combines stat cards, line chart, and bar chart in one dashboard

```

1 // Live weather dashboard for Danish cities using Open-Meteo API
2 ▾ let cities = [
3   { name: "Copenhagen", lat: 55.6761, lon: 12.5683 },
4   { name: "Aarhus", lat: 56.1629, lon: 10.2039 },
5   { name: "Odense", lat: 55.4038, lon: 10.4024 },
6   { name: "Aalborg", lat: 57.0488, lon: 9.9217 }
7 ];
8 let weatherData = [];
9 let loaded = false;
10 let animProgress = 0;
11
12 ▾ function setup() {
13   createCanvas(500, 420);

```



Narration

Infographic design tips

- **Hierarchy:** biggest/boldest element = most important message
- **Consistency:** pick 3-4 colours and stick with them
- **White space:** don't fill every pixel — let the layout breathe
- **Flow:** guide the eye from top to bottom, left to right
- **Source:** always credit your data source

Narration

Data poster

Class Assignment: Data Poster

Using the dataset from the previous assignment:

1. Sketch a layout on paper: where will the title, key numbers, and charts go?
2. Build an infographic in p5.js that combines at least two different visual elements (e.g. chart + stat boxes, pictogram + progress bars)
3. Add a title, source credit, and consistent colour scheme
4. Add animation or interaction to make it feel alive



Narration

Summary

- Data visualisation = mapping values to visual properties with `map()`
- Bar charts: `rect()` with height from data
- Line charts: `line()` segments between consecutive data points
- Scatter plots: `circle()` with x and y from two variables
- Pie charts: `arc()` with angles from data proportions
- Infographics combine multiple elements into one cohesive layout

Narration